

Nikolas Varga

Sound, Motion and Computer Interaction Final Reflection

Professor Aston McCullough

December 8, 2025

Computer Interaction

In our project “The Augmenter”, we investigate the power dynamics between human performer and virtual music system using sound and motion parameters. The power relationship between musicians or musicians and composers is present in all musical groups. For example, in classical orchestra performances, there exists a tension between the conductor’s definition of timing, the composition as read by the musicians, and the composer’s original intent with the piece. As pieces grow older, the interpretation of the piece can also change, informed by the change in practices and musical culture overall. The system was developed by myself, Nikolas Varga, Kentaro Kasai, Zoe Mumford, Mar Munne, and Amelie Pollack

Our system is informed by the musical philosophy of John Cage, who proposes “a more meaningful term” in the “organization of sound” (Kostelanetz & Cage, 1974). We do not use traditional definitions for virtual instrument or composer, instead choosing to resample (organize) and augment selected sound from the user’s vocal input and their chosen backing track. We address the key technical challenges of music information retrieval, object detection using computer vision, audio augmentation, and live input and output streams. Through the integration of motion parameters, we seek to question the limits of embodiment, defined by Preston-Dunlop and Sanchez-Colborn (Preston-Dunlop & Sánchez-Colberg, 2002). Finally, we use indeterminacy in the Cageian sense. Andrew Culver, his colleague, stated “[Cage] liked to use chance operations

because it emphasized that he was asking questions,” (Bernstein & Hatch, 2001). Our questions are as follows:

What happens when the agency of an interactive computer music system and its user are balanced, such that they fight for control?

What might this mean for an AI-focused future in audio and video processing, where the “black box” of processing becomes increasingly opaque?

How are indeterminacy and agency related in an interactive computer music system?

“The Augmenter” is designed to be used by acapella performers. The virtual listener calculates music information retrieval (MIR) data, while the composer uses motion tracking and randomness to determine augmentations on the audio and vary the character of the system across performances. The virtual composer and instrument are grouped together, incorporating indeterminacy informed by the virtual listener and video parameters to perturbate the acapella performance. We bring the agency of the computer to the forefront of the system design by applying sudden and overpowering levels of augmentation and distortion on the audio signals to the degree that they disturb the regular flow of the performer.

An interactive computer system involves action from both the computer and the user. In the interactive system, there should be “virtual” music tools within the bounds of the computer. This includes virtual performers, virtual listeners, virtual composers, and virtual instruments (Winkler, 2001). Our virtual listener uses MIR, while our virtual composer and instrument are formed by the indeterminacy found in the augmentations we apply to generate delayed and resampled audio. A point of inquiry is the tension produced in the “battle” for control between the computer and the user. When our system plays back inharmonic and unsettling augmentations of the user’s voice

and backing track, it is “creating a power relationship between the [user and computer],” (Winkler, 2001).

In interactive dance, the term “interaction” must be defined precisely and differentiated from the way dancers and musicians have been interacting for centuries. Indeterminacy makes the system interactive rather than automatic. Any interactive system should be able to respond to a performer’s actions in “predictable or unpredictable ways *without the need for a performer to direct it*,” (Toenjes, 2007). The interactive performance is understood to be a collaboration between the composer and dancer, and perhaps blurs those roles, where “one actually creates the other, both formally and materially, yet where an independence of one from the other is possible and even desirable from time to time,” (Toenjes, 2007). Interaction should make the system’s performance unique across performances. “It is now possible to... join the mechanical power of the machine to the nuanced and ‘subjectivizing’ control of the human performer,” (Toenjes, 2007). In our system, “interaction” occurs between the user and computer system. By using indeterminacy to determine when and how certain augmentations are produced, we introduce a level of subjectification to the computer, balancing the system’s agency with that of the user.

We use MIRtoolbox to analyze the summed vocal and backing track input. We calculate mirinharmonicity, mirroughness, and mirbrightness, which affect the augmentation from MusicAugmenter. We identify these as pertinent to how out of tune the singer is with the backing track. “An estimation of the total roughness is available in mirroughness by computing the peaks of the spectrum, and taking the average of all the dissonance between all possible pairs of peaks (Sethares, 1998),” (Lartillot et al., 2008). Figure 1 demonstrates the measurement of roughness, corresponding approximately to detune between two voices of an oscillator, for example.

Brightness measures the amount of high frequency energy in a signal beyond a cutoff frequency of 1500 Hz (Oliver Lartillot, 2021).

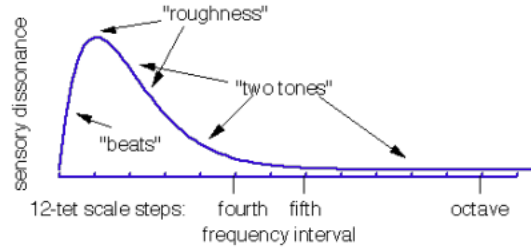


Fig. 1: Visualization of roughness, as defined by MIRToolbox (Oliver Lartillot, 2021).

Motion

As for motion, we use the YOLO v4 Tiny model for person detection, extracting keypoints using the hrnetObjectKeypointDetector. In general, the image is treated as 2 dimensional array of pixels, with intensity $\in [0,255]$ once it is converted to grayscale (Parekh, 2021). We first calculate the difference image from the current and previous frame using Equation 1. This creates a black and white binary mask where pixels take value 255 where motion has occurred, and 0 everywhere else.

$$diffImage = |frame - prevFrame| > sensitivity$$

Eq. 1: Binary mask calculation.

From this, we detect motion by finding if there are non-zero elements along the x-dimension of the difference mask. If a person is present and classified, we use the bounding box around the person to proceed. We calculate the centroid using equation 2, then use a Kalman filter for motion smoothing. The Kalman filter uses “a series of measurements observed over time and produces estimates of unknown variables by calculating a joint probability distribution over the variables for each timeframe (Kalman, 1960),” (Parekh, 2021). We use previous values of the centroid to predict the next centroid value, which is useful if the centroid disappears between frames. If the

person finder is not used to classify people, the bounding box is determined using the difference image and corrected using the Kalman filter.

$$centroid = round \left(\left[x_{0,bbbox} + \frac{w_{bbbox}}{2}, y_{0,bbbox} + \frac{h_{bbbox}}{2} \right] \right)$$

Equation 2: Centroid calculation.

In the backend, we visualize and track the centroid's center position. We pass this information to the audio app using a binary file system, developed by Kentaro Kasai. A binary file was used because it enables real-time information synchronization without the concern of synchronous reads and writes causing file corruption. Once the bounding box exceeds a certain pixel width threshold, a delay effect is applied in the audio app. This section was develop by Mar Munne.

Sound

In the audio app, codeveloped by Zoe Mumford, Amelie Pollack, and I, the user can configure their desired output device, buffer size, and input device using the audio settings popup, designed by Amelie Pollack. After an audio file is loaded, the user can start the system, which can operate without video as well. Augmentation is applied using a MusicAugmenter object for MIR parameters and an audioexample.Echo object for video parameters.

I designed the MusicAugmenter object, which applies multiplicative noise and resampling to the incoming signal depending on the value of the MIR parameters. The object is set up in a similar way to MATLAB's DSP Toolbox objects, where using the object.step() method generates a frame of audio. Using the step method also stores the frame in the object's .a member, which is used for resampling. This stores the previous n seconds of audio, where n is chosen by the user.

If roughness is greater than 10, the signal is amplitude modulated using white noise, resulting in a rougher audio signal in general (Tarr, 2019). This masks the background track for the user, which can lead to self-reinforcing distortion or inharmonicity through inaccurate harmonization. The noise is generated using a `dsp.ColoredNoise` object, which is scaled using equation 3. The noisy signal is normalized.

$$noise = whiteNoise * mapped\ roughness \in [0,1] * noiseScale$$

$$noisy\ audio = noise.* audio \in \mathbb{R}^{buffer\ size \times 1}$$

Eq. 3: Noise scaling and amplitude modulation equation

The resampling occurs infrequently, since it has a similar effect to the delay. During a given frame, if inharmonicity is greater than the historical maximum inharmonicity value, there is a 1/10 chance for resampling. The `resample()` method divides what is present in the `.a` member into `n` parts, picking random starting and ending samples that the resampled audio is between. The system then adds a delay effect to the resample depending on a random outcome. The system runs 8 coin flips with a probability=0.6 of heads and 0.4 of tails. For each heads, the system applies a delay effect (from where?) with feedback level mapped from `nHeads` $\in [0,8]$ to a range $[0, 0.5]$. This is applied `nHeads` times.

The MIR parameters are updated every 6 seconds. These are packed into an `mirStruct` (designed by me) which contains fields for each MIR parameter we calculate and passed to the `MusicAugmenter` in the audio app. This limits how often the resampling can occur. Future work would untie the resampling from maximum value checking, using a binary threshold between the current maximum and minimum instead.

References

- Bernstein, D. W., & Hatch, C. (Eds.). (2001). Writings through John Cage's music, poetry, and art: Proceedings from the conference entitled "Here Comes Everybody: The Music, Poetry, and Art of John Cage" which took place at Mills College in Oakland, Calif. from November 15 to 19, 1995. Conference Entitled Here Comes Everybody: The Music, Poetry, and Art of John Cage, Chicago. The University of Chicago Press.
- Kostelanetz, R., & Cage, J. (1974). John Cage. Lane.
- Lartillot, O., Toiviainen, P., & Eerola, T. (2008). A Matlab Toolbox for Music Information Retrieval. In C. Preisach, H. Burkhardt, L. Schmidt-Thieme, & R. Decker (Eds.), *Data Analysis, Machine Learning and Applications* (pp. 261–268). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-78246-9_31
- Oliver Lartillot. (2021). MIRtoolbox 1.8.1 User's Manual [User's Manual]. University of Oslo, Norway.
- Parekh, R. (2021). *Fundamentals of image, audio, and video processing using MATLAB: With applications to pattern recognition* (First edition). CRC Press.
- Preston-Dunlop, V., & Sánchez-Colberg, A. (2002). *Dance and the performative: A choreological perspective -Laban and beyond*. Verve.
- Tarr, E. (2019). *Hack audio: An introduction to computer programming and digital signal processing in MATLAB®*. Routledge.
- Toenjes, J. (2007). Composing for Interactive Dance: Paradigms for Perception. *Perspectives of New Music*, 45(2), 28–50. JSTOR.
- Winkler, T. (2001). *Composing interactive music: Techniques and ideas using Max* (Paperback edition). The MIT Press.

